6.

Fragmento	Operaciones aproximadas	Big O	Justificación
Acceder a vector[5]	1	O(1)	solo consulta una posición, sin importar el tamaño de la lista
Recorrer un arreglo	n	O(n)	el ciclo pasa una vez por cada elemento
Dos ciclos anidados	$n \times n = n^2$	$O(n^2)$	bucles anidados, se multiplican entre si
Dividir n entre 2 repetidamente	log n	O(log n)	en cada paso se reduce a la mitad.
Ciclo n + reducción logarítmica	$n \times \log n$	O(n log n)	se divide a la mitad y al final une todo

7.

el algoritmo “Algoritmo A – O(n)” escala mejor, debido a que únicamente debe recorrer cada elemento una vez, mientras que el “Algoritmo B – O(n²)” cuadriplica los pasos, por que tiene bucles “for” anidados, los cuales, en pocas palabras hace que se multiplique las lista entre sí

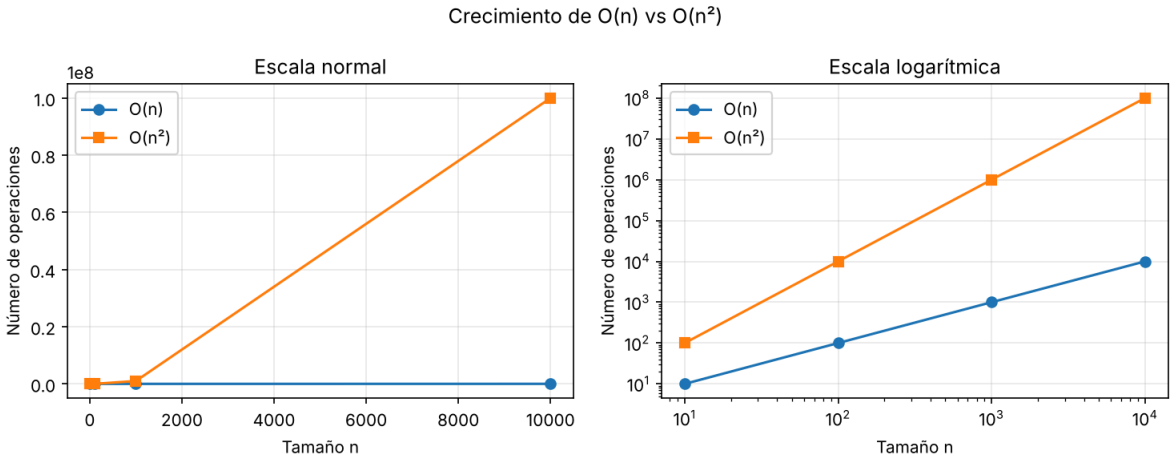
10.

Tamaño n	Tiempo O(n)	Tiempo O(n2)	¿Cuál fue más rápido?
100	24880	209650	O(n)
1.000	29660	4233470	O(n)
5.000	123950	98694327	O(n)
10.000	249320	300444309	O(n)

11.

n	O(1)	O(log n) aprox.	O(n)	O(n log n) aprox.	O(n2)
10	1	3	10	33	100
100	1	7	100	664	10.000
1.000	1	10	1.000	9.966	1.000.000
10.000	1	13	10.000	132.877	100.000.000

12.



En la gráfica con escala normal se ve que O(n) casi parece una línea pegada abajo, mientras que O(n^2) se dispara hacia arriba. Con n = 10.000, O(n) hace 10.000 operaciones y O(n^2) hace 100.000.000.

13.

1. ¿Qué diferencia existe entre tiempo de ejecución y complejidad Big O?

El tiempo de ejecución es lo que realmente demora el programa en segundos o menos y cambia según la potencia del pc. Mientras que Big O es cómo crece el número de operaciones cuando aumenta el tamaño de los datos

2. ¿Por qué O(1) se considera una complejidad eficiente?

Porque el trabajo es siempre el mismo osea 1, sin importar si hay millones de datos. Por ejemplo, acceder a datos[0] siempre es una sola operación

3. ¿Qué ocurre con un algoritmo O(n) cuando duplicamos el tamaño de entrada?

El trabajo también se duplica, porque crece de igual manera que n

4. ¿Qué ocurre aproximadamente con un algoritmo O(n^2) cuando duplicamos n?

El trabajo se multiplica por 4, porque $(2n)^2 = 4n^2$

5. ¿Por qué los ciclos anidados suelen relacionarse con O(n^2)?

Porque por cada vuelta del ciclo externo (n veces) el ciclo interno vuelve a dar n vueltas, así que se hacen n × n = n^2 operaciones

6. ¿Qué característica permite reconocer un algoritmo O(log n)?

Que en cada paso se saca mitad

7. ¿Cuál de los dos algoritmos experimentales presentó mayor crecimiento?

El cuadrático $O(n^2)$. En mi tabla, el lineal pasó de 6.380 ns (n = 100) a 123.805 ns (n = 10.000), unas 19 veces más. El cuadrático pasó de 111.279 ns a 391.922.479 ns, miles de veces más

8. ¿Los resultados medidos fueron exactamente iguales entre las diferentes ejecuciones? ¿Por qué?

No. Ejecuté el programa 3 veces y los tiempos cambiaron cada vez porque el tiempo depende de la pc, los otros programas abiertos y de Java

9. Si tuviera que procesar diez millones de registros, ¿qué tipo de complejidad preferiría?

Preferiría O(log n) para ir cortando el número de operaciones a la mitad, siendo más eficaz

10. ¿Por qué un ingeniero de software debe analizar la eficiencia antes de seleccionar un algoritmo?

Porque un programa siempre funciona bien con pocos dato, en cambio con muchos se vuelve lento. Por eso debemos saber seleccionar el algoritmos, para evitar problemas de rendimiento y ahorrar tiempo

El reto adicional ¿Es $O(n^2)$?

No. Los dos ciclos están uno después del otro , no uno dentro del otro. El primero hace n operaciones y el segundo hace otras n. Si estuvieran anidados (uno dentro de otro) si sería "$O(n^2)$" por que en pocas palabras se multiplicaran entre si.

Conclusión

En esta actividad aprendí que no basta con que un programa funcione; también hay que revisar qué pasa cuando se intoducen muchos datos. Con el experimento vi que el algoritmo lineal O(n) casi no cambia al aumentar n, mientras que el cuadrático $O(n^2)$ se demora muchísimo más . También entendí que Big O no mide segundos exactos sino la tendencia de crecimiento, y que los tiempos medidos cambian en cada ejecución. Por eso, para una aplicación que procese millones de registros, elegiría un algoritmo O(n) u O(n log n) y evitaría los ciclos anidados, por que son muy demorados.